

# Zero to Hero Study Handbook: offline-ollama-rag-app

## Module 1: Foundations & Architecture

### 1.1 What this project does

`offline-ollama-rag-app` is a local-first Retrieval-Augmented Generation (RAG) application that runs entirely against local Ollama models.

Core capabilities in this repository: 1. Build a vector index from local markdown/text knowledge files. 2. Answer questions in two modes: 3. Baseline mode (no retrieval context). 4. RAG mode (retrieve relevant chunks, then generate with context). 5. Evaluate baseline vs RAG with keyword recall metrics. 6. Expose the same runtime through CLI, Streamlit UI, and tutorial notebooks.

Main use cases: 1. Offline question answering over internal policy/runbook documents. 2. Controlled comparison of retrieval impact using reproducible evaluation artifacts. 3. Learning-oriented RAG implementation with cleanly separated modules.

### 1.2 Core paradigms and patterns used

Definitions first, then where they appear in code:

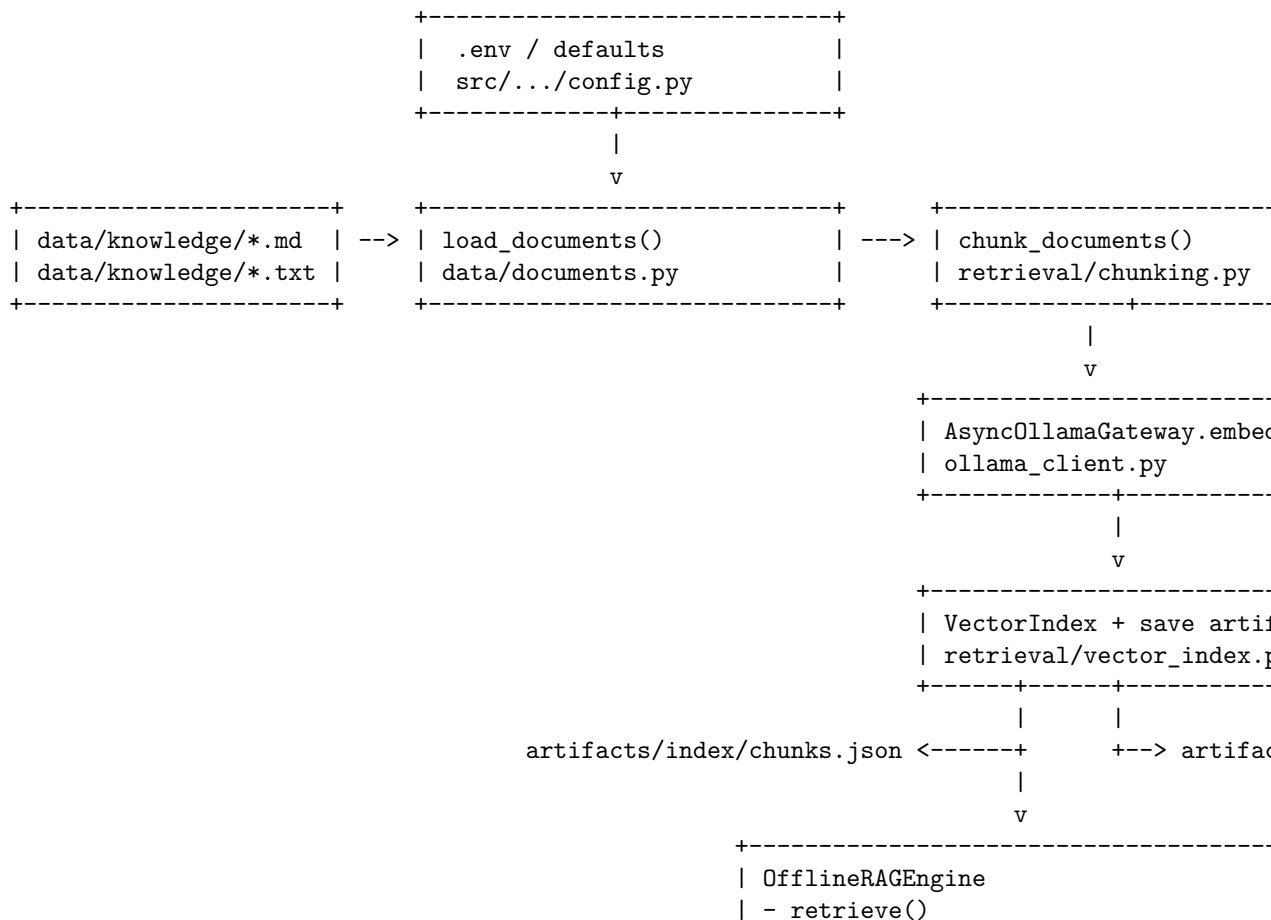
1. Retrieval-Augmented Generation (RAG): The model receives retrieved local context snippets before generating an answer. Code: `src/offline_ollama_rag/retrieval/rag_engine.py` (`OfflineRAGEngine.answer_with_rag`).
2. Baseline-vs-RAG evaluation pattern: The same question set is answered twice: once without retrieval, once with retrieval; metrics compare both. Code: `src/offline_ollama_rag/eval/evaluator.py` (`run_evaluation`).
3. Async I/O orchestration: Model calls are asynchronous to avoid blocking on network/API I/O with local Ollama server. Code: `src/offline_ollama_rag/ollama_client.py`, `src/offline_ollama_rag/pipeline.py`.
4. Typed schema contracts: Pydantic models define and validate document, chunk, and evaluation row/summary shapes. Code: `src/offline_ollama_rag/schemas.py`.
5. Layered architecture: Configuration, data loading, retrieval, model gateway, pipeline orchestration, and interfaces (CLI/UI) are separated. Code: `config -> data/retrieval/eval/reporting -> pipeline -> cli/app`.
6. Functional core + thin interface shells: Core logic lives in pure or mostly pure functions/classes; CLI and Streamlit wrap those functions. Code:

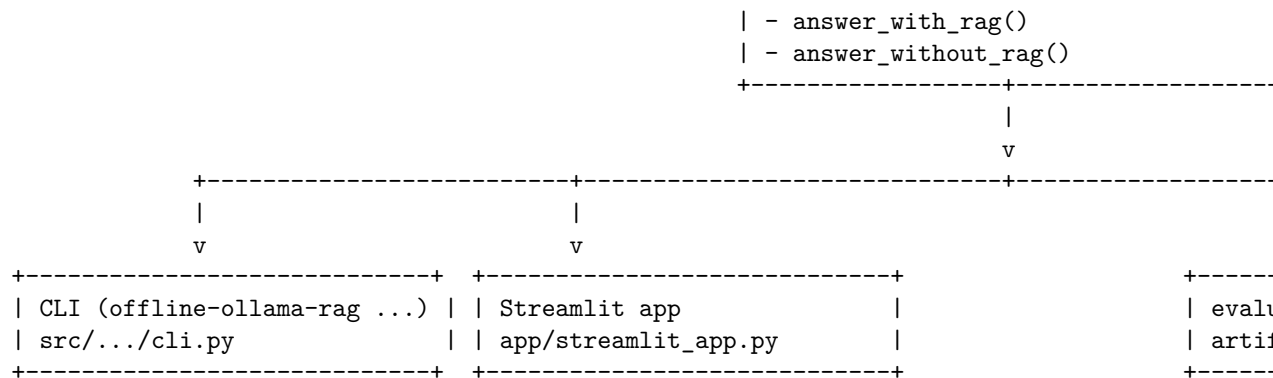
src/offline\_ollama\_rag/pipeline.py, src/offline\_ollama\_rag/cli.py,  
app/streamlit\_app.py.

### 1.3 Architecture overview

Primary components: 1. Settings (config.py): central runtime config from .env and defaults. 2. load\_documents (data/documents.py): reads .md/.txt knowledge docs. 3. chunk\_documents (retrieval/chunking.py): creates overlapping word windows. 4. AsyncOllamaGateway (ollama\_client.py): embeds text and chats with local models. 5. VectorIndex (retrieval/vector\_index.py): cosine similarity retrieval. 6. OfflineRAGEngine (retrieval/rag\_engine.py): retrieval + prompting + fallback logic. 7. pipeline.py: orchestration for index build, Q&A, evaluation, and run-all. 8. eval/evaluator.py + reporting/reporting.py: metrics + output artifacts. 9. cli.py + app/streamlit\_app.py: user-facing interfaces.

ASCII architecture diagram:





## Module 2: Repository Map

Focus files a new contributor should read first:

| File/Directory Path              | Primary Responsibility                                         | Key Classes/Functions                                                                      | Important Configs/Variables                                                 |
|----------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| README.md                        | Project overview, setup, and CLI usage examples                | N/A                                                                                        | Model names, command sequence                                               |
| pyproject.toml                   | Packaging, dependencies, lint/type/test config, CLI entrypoint | [project.scripts] Python version<br>offline-ollama-rag<br>= offline_ollama_rag_cli/main.py | Python version range, dependency list, Build/Make/Pytest config             |
| .env.example                     | Runtime environment template                                   | N/A                                                                                        | OLLAMA_HOST, CHAT_MODEL, EMBEDDING_MODEL, re-retrieval/generation/path keys |
| src/offline_ollama_rag/config.py | Envrionment backed settings and path resolution                | Settings, get_settings, ensure_dirs, resolve                                               | All runtime settings and resolved path properties                           |
| src/offline_ollama_rag/schema.py | Types/schemas for models                                       | Document, DocumentChunk, RetrievedChunk, QAExample, EvalPrediction, EvalSummary            | Field names and constraints                                                 |

| File/Directory Path                         | Primary Responsibility                     | Key Classes/Functions                                                                                                                                                                                   | Important Con-figs/Variables                                                                   |
|---------------------------------------------|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| src/offline_ollama_krag/data/documents      | loading and title extraction               | <code>load_documents</code> , <code>_extract_title</code>                                                                                                                                               | <code>SUPPORTED_SUFFIXES = {".md", ".txt"}</code>                                              |
| src/offline_ollama_krag/retrieval/chunking  | chunking with overlap                      | <code>chunk_documents</code> , <code>chunking_document</code>                                                                                                                                           | <code>chunk_size_words</code> , <code>chunk_overlap_words</code> , computed <code>step</code>  |
| src/offline_ollama_krag/retrieval/vector    | retrieval + persistence                    | <code>VectorIndex</code> , <code>search</code> , <code>build_vector_index</code> , <code>save_vector_index</code> , <code>load_vector_index</code>                                                      | Embedding matrix shape, alignment checks                                                       |
| src/offline_ollama_krag/wrapper             | client over Ollama SDK                     | <code>AsyncOllamaGateway</code> , <code>ensure_required_models</code> , <code>embed_texts</code> , <code>chat</code>                                                                                    | <code>ModelNames</code> , <code>models</code> , <code>timeout</code> , <code>parameters</code> |
| src/offline_ollama_krag/retrieval/rag       | Base retrieval RAG answer generation       | <code>OfflineRAGEngine</code> , <code>RAG_SYSTEM_PROMPT</code> , <code>answer_with_rag</code> , <code>answer_without_rag</code> , <code>_build_context_block</code> , <code>_extractive_fallback</code> | <code>BASELINE_SYSTEM_PROMPT</code>                                                            |
| src/offline_ollama_krag/eval/evaluation     | Keyword recall loading, scoring, and loop  | <code>load_eval_examples</code> , <code>keyword_recall</code> , <code>nonempty_rate</code> , <code>run_evaluation</code>                                                                                | Keyword recall <code>math</code> , non-empty rate <code>math</code>                            |
| src/offline_ollama_krag/reporting/reporting | Base reporting outputs and markdown report | <code>reporting_options</code> , <code>save_summary</code> , <code>render_report</code>                                                                                                                 | <code>_REPORT_TEMPLATE</code> , output file conventions                                        |
| src/offline_ollama_krag/rag/pipeline.py     | runtime orchestration                      | <code>build_index</code> , <code>load_index</code> , <code>answer_question</code> , <code>evaluate</code> , <code>run_all</code>                                                                        | Return payload shapes and artifact paths                                                       |
| src/offline_ollama_krag/rag/CLI.py          | commands and async execution bridge        | <code>build_index_cmd</code> , <code>ask_cmd</code> , <code>evaluate_cmd</code> , <code>run_all_cmd</code> , <code>serve_app_cmd</code> , <code>_run_async</code>                                       | <code>ask</code> flag <code>rag: bool = True</code> , <code>serve-app</code> default port 8503 |
| app/streamlit_app.py                        | Chat UI for local Q&A                      | Streamlit top-level flow + <code>asyncio.run(answer_questions...)</code>                                                                                                                                | Sidebar <code>use_rag</code> toggle, session state                                             |
| scripts/execute_notebooks.py                | Execute trial notebooks in sequence        | <code>NOTEBOOKS</code> , <code>execute_notebook</code> , <code>main</code>                                                                                                                              | Notebook <code>timeout 1200</code> seconds                                                     |

| File/Directory Path                        | Primary Responsibility                           | Key Classes/Functions                                                                                                                             | Important Con-figs/Variables                                                                                      |
|--------------------------------------------|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>scripts/generate_notebooks.py</code> | Typically generate tutorial notebooks            | <code>_write_notebook</code> , <code>main</code>                                                                                                  | Notebook filenames and tutorial cell content                                                                      |
| <code>tests/</code>                        | Expected behavior and invariants                 | <code>test_load_documents_overlap</code> , <code>test_chunk_documents_overlap</code> , <code>test_vector_search_returns_hit</code> , metric tests | Overlap math, chunking, metrics                                                                                   |
| <code>data/knowledge/</code>               | Local knowledge corpus used for indexing         | Markdown content files                                                                                                                            | Source docs used to answer questions                                                                              |
| <code>data/eval/questions.json</code>      | Eval JSON dataset for baseline-vs-RAG comparison | JSON array of <code>QAExample</code> -shaped objects                                                                                              | <code>question_id</code> , <code>question</code> , <code>reference_answer</code> , <code>required_keywords</code> |
| <code>artifacts/</code>                    | Persisted outputs from pipeline runs             | <code>index</code> , <code>evals</code> , <code>reports</code> , <code>run_summary.json</code>                                                    | File outputs consumed by app and analysis                                                                         |

## Module 3: Core Execution Flows

This module walks through main runtime paths with exact function names and data shapes.

### 3.1 Flow A: Build index (offline-ollama-rag build-index)

Entry point chain: 1. CLI command `build_index_cmd()` in `src/offline_ollama_rag/cli.py`. 2. Calls `build_index(settings, gateway)` in `src/offline_ollama_rag/pipeline.py`.

Step-by-step: 1. `gateway.ensure_required_models(chat_model, embedding_model)` validates local model presence. 2. `load_documents(settings.resolved_knowledge_dir)` loads `.md/.txt`. 3. `chunk_documents(...)` creates `DocumentChunk` objects. 4. `gateway.embed_texts(embedding_model, [chunk.text...], timeout_seconds=...)` gets embedding matrix. 5. `build_vector_index(chunks, embeddings)` validates row alignment. 6. `save_vector_index(index, settings.embeddings_file, settings.chunks_file)` writes. 7. `artifacts/index/embeddings.npy` 8. `artifacts/index/chunks.json` 9. Returns an info dict with document/chunk counts and artifact paths.

Short code fragment (from `pipeline.py`):

```
docs = load_documents(settings.resolved_knowledge_dir)
chunks = chunk_documents(documents=docs, chunk_size_words=..., chunk_overlap_words=...)
embeddings = await gateway.embed_texts(settings.embedding_model, [chunk.text for chunk in ch
```

```
index = build_vector_index(chunks=chunks, embeddings=embeddings)
save_vector_index(index, settings.embeddings_file, settings.chunks_file)
```

Output shape from build\_index:

```
{
  "n_documents": 5,
  "n_chunks": 5,
  "embedding_dimension": 768,
  "embeddings_file": ".../artifacts/index/embeddings.npy",
  "chunks_file": ".../artifacts/index/chunks.json"
}
```

### 3.2 Flow B: Ask question (offline-ollama-rag ask)

Entry point: 1. CLI ask\_cmd(question: str, rag: bool = True) in cli.py. 2. Calls answer\_question(settings, question, use\_rag=rag) in pipeline.py.

Branch behavior: 1. Shared setup: 2. AsyncOllamaGateway(settings.ollama\_host) 3. ensure\_required\_models(...) 4. load\_index(settings) reads persisted index artifacts. 5. OfflineRAGEngine(settings, gateway, index) initialized. 6. If use\_rag=True: 7. engine.answer\_with\_rag(question): 8. retrieve(question) embeds question and calls VectorIndex.search(top\_k=settings.retrieval\_top\_k). 9. \_build\_context\_block(retrieved) creates formatted context lines. 10. gateway.chat(...) generates answer with RAG\_SYSTEM\_PROMPT. 11. On timeout: \_extractive\_fallback(retrieved) returns top chunk text. 12. If use\_rag=False: 13. engine.answer\_without\_rag(question) sends raw question with BASELINE\_SYSTEM\_PROMPT. 14. On timeout: returns "I don't know.".

RAG result payload shape (answer\_question return):

```
{
  "mode": "rag",
  "question": "How long are raw event logs retained?",
  "answer": "...",
  "retrieved": [
    {
      "score": 0.9876,
      "source": "data_policy.md",
      "chunk_id": "data_policy.md::chunk-0",
      "text": "# Data Governance Policy ..."
    }
  ]
}
```

Baseline payload shape:

```
{
  "mode": "baseline",
  "question": "How long are raw event logs retained?",
  "answer": "...",
  "retrieved": []
}
```

### 3.3 Flow C: Evaluation (offline-ollama-rag evaluate)

Entry point: 1. CLI `evaluate_cmd()` in `cli.py`. 2. Calls `evaluate(settings)` in `pipeline.py`.

Step-by-step: 1. Build gateway, ensure models, load index, build OfflineRAGEngine. 2. Load questions with `load_eval_examples(settings.resolved_evaluation_file)`. 3. Run `run_evaluation(engine, examples)`: 4. For each `QAExample`, compute: 5. `baseline_answer = await engine.answer_without_rag(...)` 6. `rag_answer, _ = await engine.answer_with_rag(...)` 7. `baseline_keyword_recall = keyword_recall(...)` 8. `rag_keyword_recall = keyword_recall(...)` 9. Aggregate means and non-empty rates into `EvalSummary`. 10. Persist outputs: 11. `save_predictions(rows, artifacts/evals/predictions.csv)` 12. `save_summary(summary, artifacts/evals/summary.json)` 13. `render_report(..., artifacts/reports/offline_ollama_rag_report.md)`

Input shape from `data/eval/questions.json` (`QAExample`):

```
{
  "question_id": "q1",
  "question": "What paging system handles the primary on-call rotation?",
  "reference_answer": "Solar Pager handles primary on-call.",
  "required_keywords": ["solar pager"]
}
```

Prediction row shape (`EvalPrediction`):

```
{
  "question_id": "q1",
  "question": "...",
  "reference_answer": "...",
  "baseline_answer": "...",
  "rag_answer": "...",
  "baseline_keyword_recall": 0.0,
  "rag_keyword_recall": 1.0
}
```

Summary shape (`EvalSummary`):

```
{
  "n_questions": 6,
  "baseline_keyword_recall_mean": 0.0,

```

```

"rag_keyword_recall_mean": 1.0,
"keyword_recall_gain": 1.0,
"baseline_nonempty_rate": 1.0,
"rag_nonempty_rate": 1.0
}

```

### 3.4 Flow D: Full pipeline (offline-ollama-rag run-all)

Entry point: 1. CLI `run_all_cmd()` in `cli.py`. 2. Calls `run_all(settings)` in `pipeline.py`.

What it does: 1. `build_index(settings, gateway)` first. 2. `evaluate(settings)` second. 3. Combines both into one payload. 4. Writes `artifacts/run_summary.json`.

Returned payload keys: 1. `chat_model` 2. `embedding_model` 3. `index` (build-index payload) 4. `evaluation` (evaluate payload) 5. `run_summary_path`

### 3.5 Flow E: Streamlit runtime path (offline-ollama-rag serve-app)

Entry point: 1. CLI `serve_app_cmd(port: int = 8503)` in `cli.py`. 2. Launches `app/streamlit_app.py`.

UI flow in `streamlit_app.py`: 1. `settings = get_settings()` 2. Show model and path info in title/sidebar. 3. `load_index(settings)` guard: 4. if missing, show error and `st.stop()`. 5. Persist chat history in `st.session_state.messages`. 6. On user prompt: 7. call `payload = asyncio.run(answer_question(settings, question=prompt, use_rag=use_rag))` 8. render `payload["answer"]` 9. render `payload["retrieved"]` inside expander when present.

### 3.6 Key data structures to memorize

Document fields: 1. `source: str` 2. `title: str` 3. `text: str`

DocumentChunk fields: 1. `chunk_id: str` 2. `source: str` 3. `title: str` 4. `text: str` 5. `start_word: int` 6. `end_word: int`

RetrievedChunk fields: 1. `chunk: DocumentChunk` 2. `score: float`

Index artifact files: 1. `artifacts/index/embeddings.npy`: float matrix shape `[n_chunks, embedding_dim]` 2. `artifacts/index/chunks.json`: array of DocumentChunk payloads aligned row-wise with embeddings

## Module 4: Setup & Run Guide

### 4.1 Prerequisites on a clean machine

1. Python 3.12.10 (from `README.md` and `pyproject.toml`).
2. uv for environment and dependency management.



3. Ollama installed and local server reachable at `OLLAMA_HOST` (default `http://127.0.0.1:11434`).
4. Required local models:
5. `embeddinggemma:latest`
6. `granite4.1:3b`

## 4.2 Installation sequence

```
git clone https://github.com/pypi-ahmad/offline-ollama-rag-app.git
cd offline-ollama-rag-app
uv python pin 3.12.10
uv sync --dev
cp .env.example .env
```

Pull required models:

```
ollama pull embeddinggemma:latest
ollama pull granite4.1:3b
```

## 4.3 Environment variables and config keys

Defined in `.env.example` and loaded by `Settings` in `config.py`:

1. Ollama connectivity and model selection:
2. `OLLAMA_HOST`
3. `CHAT_MODEL`
4. `EMBEDDING_MODEL`
5. Retrieval and generation controls:
6. `SEED`
7. `CHUNK_SIZE_WORDS`
8. `CHUNK_OVERLAP_WORDS`
9. `RETRIEVAL_TOP_K`
10. `GENERATION_TEMPERATURE`
11. `GENERATION_MAX_TOKENS`
12. `GENERATION_TIMEOUT_SECONDS`
13. `EMBEDDING_TIMEOUT_SECONDS`
14. Data and artifact paths:
15. `KNOWLEDGE_DIR`
16. `EVALUATION_FILE`
17. `ARTIFACTS_DIR`
18. `INDEX_DIR`
19. `EVAL_DIR`
20. `REPORT_DIR`

Important behavior: 1. `SettingsConfigDict(case_sensitive=False)` means env var names are case-insensitive. 2. `get_settings()` calls `ensure_dirs()`, so artifact directories are auto-created.

## 4.4 Typical command sequences

Build index:

```
uv run offline-ollama-rag build-index
```

Ask question with RAG:

```
uv run offline-ollama-rag ask "What is the enterprise first-response SLA?"
```

Ask question without retrieval:

```
uv run offline-ollama-rag ask "What is the enterprise first-response SLA?" --no-rag
```

Run evaluation:

```
uv run offline-ollama-rag evaluate
```

Run full pipeline:

```
uv run offline-ollama-rag run-all
```

Run Streamlit app:

```
uv run offline-ollama-rag serve-app --port 8503
```

Execute tutorial notebooks:

```
uv run python scripts/execute_notebooks.py
```

## 4.5 Data seeding, migrations, and external services

1. Database migrations: none in this repository.
2. Data seeding pattern:
3. Add .md/.txt files to data/knowledge/.
4. Update data/eval/questions.json with QAExample objects for evaluation.
5. Rebuild index to include new/updated knowledge documents.
6. External dependency:
7. Local Ollama service only (via AsyncOllamaGateway).

## 4.6 Contributor quality checks defined in repo

From .github/workflows/ci.yml: 1. uv run ruff check . 2. uv run mypy src 3. uv run pytest

# Module 5: Study Plan & Practice Exercises

## 5.1 Ordered study plan for a new learner

Recommended order: 1. README.md and .env.example to understand goals and runtime knobs. 2. src/offline\_ollama\_rag/config.py and schemas.py to learn config + data contracts. 3. src/offline\_ollama\_rag/data/documents.py and retrieval/chunking.py for data preparation. 4. src/offline\_ollama\_rag/retrieval/vector\_index.p

for retrieval mathematics and artifact format. 5. `src/offline_ollama_rag/ollama_client.py` and `retrieval/rag_engine.py` for model interaction and prompting. 6. `src/offline_ollama_rag/pipeline.py` for orchestration and output payloads. 7. `src/offline_ollama_rag/eval/evaluator.py` and `reporting/reporting.py` for metrics/reporting. 8. `src/offline_ollama_rag/cli.py` and `app/streamlit_app.py` for interface layer. 9. `tests/` last, to validate your mental model against expected behavior.

## 5.2 Practice exercises

1. Exercise: Explain exactly how a knowledge file becomes searchable by vector retrieval. Files to read: `data/documents.py`, `retrieval/chunking.py`, `ollama_client.py`, `retrieval/vector_index.py`, `pipeline.py`.
2. Exercise: Trace what happens when `offline-ollama-rag ask "... --no-rag` is executed. Files to read: `cli.py`, `pipeline.py`, `retrieval/rag_engine.py`.
3. Exercise: Write the exact JSON schema of one item in `data/eval/questions.json`. Files to read: `schemas.py`, `eval/evaluator.py`, `data/eval/questions.json`.
4. Exercise: Identify where timeout behavior is implemented and what fallback answer is returned. Files to read: `ollama_client.py`, `retrieval/rag_engine.py`.
5. Exercise: Describe how retrieval scores are calculated and ranked. Files to read: `retrieval/vector_index.py`.
6. Exercise: Explain why `chunks.json` and `embeddings.npy` must stay row-aligned. Files to read: `retrieval/vector_index.py`, `pipeline.py`.
7. Exercise: Identify all output artifacts produced by `evaluate` and `run-all`. Files to read: `pipeline.py`, `reporting/reporting.py`, `artifacts/`.
8. Exercise: In Streamlit flow, explain how chat history and retrieved context are rendered. Files to read: `app/streamlit_app.py`.

## 5.3 Solution outlines

1. Solution outline: `build_index` loads docs, chunks them, embeds chunk text, builds validated index, then saves `embeddings.npy` + `chunks.json`.
2. Solution outline: `ask_cmd` calls `answer_question(..., use_rag=False)`, which loads settings, ensures models, loads index, then calls `engine.answer_without_rag`; retrieved list is empty in payload.
3. Solution outline: Each evaluation item has keys `question_id`, `question`, `reference_answer`, `required_keywords`; this is validated into `QAExample`.
4. Solution outline: Timeout wrappers are in `AsyncOllamaGateway.embed_texts` and `AsyncOllamaGateway.chat` using `asyncio.wait_for`. In generation timeout cases, baseline returns "I don't know.", and RAG

uses `_extractive_fallback` or "I don't know based on the local knowledge base."

5. Solution outline: `VectorIndex.search` computes cosine similarity  $(\text{matrix} @ \text{query}) / (\text{matrix\_norm} * \text{query\_norm})$ , sorts descending with `np.argsort(scores)[::-1]`, and returns top-k.
6. Solution outline: `build_vector_index` raises `ValueError` if `len(chunks) != len(embeddings)`. Each embedding row must correspond to exactly one chunk metadata row.
7. Solution outline: `evaluate` writes `artifacts/evals/predictions.csv`, `artifacts/evals/summary.json`, and `artifacts/reports/offline_ollama_rag_report.md`. `run_all` additionally writes `artifacts/run_summary.json`.
8. Solution outline: Streamlit stores messages in `st.session_state.messages`; each assistant message may include a `retrieved` list shown inside `st.expander("Retrieved context")` via `st.json`.

## Understanding Checklist

Use this checklist to self-verify mastery:

1. Can you explain how `Settings` resolves relative paths and creates runtime directories?
2. Can you describe the difference between `answer_with_rag` and `answer_without_rag` end-to-end?
3. Can you state the exact fields in `DocumentChunk`, `QAExample`, and `EvalSummary`?
4. Can you explain where and how cosine similarity retrieval is implemented?
5. Can you trace the full call path from CLI command `ask` to final JSON payload output?
6. Can you explain the timeout fallback behavior for both baseline and RAG generation?
7. Can you list all artifacts produced by `build-index`, `evaluate`, and `run-all`?
8. Can you explain how keyword recall is computed and aggregated in evaluation?
9. Can you map Streamlit UI events to `pipeline.answer_question` invocation?
10. Can you modify `.env` retrieval settings and predict which modules will reflect the change?